LN - Project Report

Group 31

Salvador Florêncio — ist1102706, Tiago Silva — ist1103811, Luís Calado — ist1103883

1 Models

In our initial approach, we applied a Multinomial Naive Bayes classifier to the dataset preprocessed using the Term Frequency-Inverse Document Frequency (TF-IDF) method, based on the notebook provided in lab classes and an approach to a similar challenge found on Kaggle. By doing so, we were able to achieve an accuracy score of 37%. Then, by using the CountVectorizer method instead of TF-IDF, we achieved an accuracy of 64%.

Building on this, we explored various models, including a Support Vector Machine with a TF-IDF vectorizer, Logistic Regression, Random Forests and even experimented a deep learning model, TinyBERT [3], in an effort to achieve higher accuracy.

Feature-based models

Our best feature-based model was the Support Vector Machine with a linear kernel combined with TF-IDF Vectorizer, resulting in an accuracy of 69%, while our second-best model was Logistic Regression, also combined with TF-IDF Vectorizer (68% accuracy).

For **pre-processing**, we tried various approaches, with our best one being: lowercasing; removing punctuation, stopwords, URLs (we noticed one of the plots contains a URL), and removing actor names, as we observed that in many plots, actor names appeared alongside character names (e.g., "Charley Kittridge (Joe Dorsey)"). We also tokenized the text and used lemmatization. Stemming yielded worse results, so we discarded it.

Sequence models

To explore sequence models, we employed TinyBERT [3], a BERT [1] flavour with fewer parameters that retains the ability to learn the rich knowledge embedded in the pre-trained BERT model. The specific variant we used is `TinyBERT_General_4L_312D`, available through Hugging Face's `transformers` library. The model was fine-tuned for the task by setting up a custom PyTorch `Dataset` and using the `AutoTokenizer` to tokenize the input data. However, this transfer learning technique did not yield an improvement in accuracy (67.7%) compared to our previous models, leading us to select the SVM with TF-IDF model as our best-performing model.

2 Experimental Setup and Results

Support Vector Machine with TF-IDF

We applied a manual 90-10 test split and trained the model using 10-fold cross-validation on the training set. For the tuning of the hyperparameters, we also used `GridSearchCV` together with a pipeline to find the best combination of parameters for `TfidfVectorizer` and SVM. However, since `GridSearchCV` exhaustively searches all possible parameter combinations, it took a significant amount of time to run. As a result, we opted for `RandomizedSearchCV`, which explores only a fixed number of hyperparameter combinations.

For the SVM parameters, we used bigrams to capture word relationships and set `max_features=20000` to filter out the least relevant terms, reducing the risk of overfitting. Additionally, we enabled `sublinear_tf`, as it provided better results [2].

We opted for the linear kernel as it provided the best accuracy in our experiments, though the sigmoid kernel also yielded good results. However, the sigmoid kernel is typically suited to datasets with a sigmoidal shape, and requires careful fine-tuning to find generalizable decision boundaries. Due to its more specific use cases, the sigmoid kernel is less commonly used, which led us to favor the linear kernel for this task.

TinyBERT

An 85-15 train-test split was used to evaluate the model's performance. The training loop was structured using mixed precision for performance gains, with `GradScaler` handling floating point operations. We optimized the model using AdamW optimizer, with learning rates of {3e-5, 5e-5, 1e-4}. The model was trained for {3, 4} epochs with a batch size of {16, 32}, using cross-entropy loss as the loss function. During each epoch, we truncated the input sequences to TinyBERT's maximum token limit of 512 (see Discussion) and ran forward and backward passes to calculate the loss and update the model weights accordingly.

For evaluation, we transitioned the model to evaluation mode and computed the accuracy on the validation set. This fine-tuning approach allowed us to achieve an accuracy no higher than 67.7%.

Results

From the recall scores for each label, it is clear that they are quite inconsistent. For instance, the recall for the "Western" genre is 0.91, while for "Sci-Fi" genre it is only 0.40. This gap is primarily due to the challenging nature of the task, even for human annotators, and also because of the fact that the dataset is unbalanced (genre distribution[1]). The precision ranges from 0.55, for "Drama", up to 0.95, for "Western".

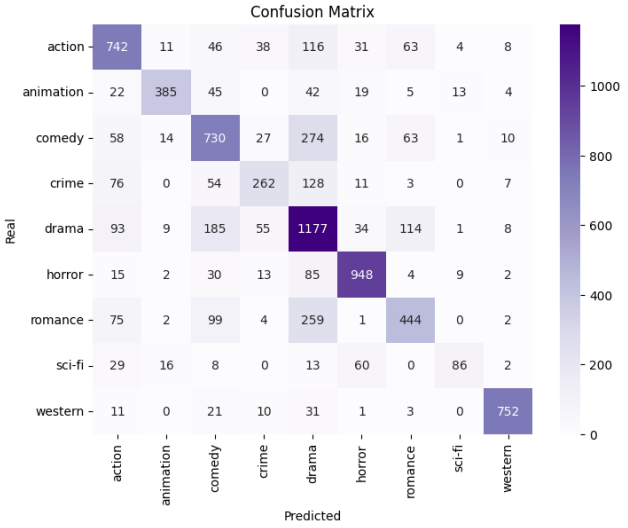


Figure 1: Confusion matrix with SVM with TF-IDF

Table 1: Evaluation metrics with SVM with TF-IDF

Label	Precision	Recall	F1-Score	Support
Action	0.66	0.70	0.68	1059
Animation	0.88	0.72	0.79	535
Comedy	0.60	0.61	0.61	1193
Crime	0.64	0.48	0.55	541
Drama	0.55	0.70	0.62	1676
Horror	0.85	0.86	0.85	1108
Romance	0.64	0.50	0.56	886
Sci-fi	0.75	0.40	0.52	214
Western	0.95	0.91	0.93	829

3 Discussion

Right from the beginning, we recognized that predicting movie genres (with 9 labels...) from the plot would be a challenging endeavor, even for a human classifier.

On one hand, some movie plots were very short (word counts in plot descriptions [2]), which made it even more difficult for our model to accurately predict their genre. On the other hand, a significant number of movie plots exceeded 512 words and were truncated by TinyBERT, as the model only accepts up to 512 tokens.

The research was limited to only movies from the given `train.txt` dataset consisting of 8041 movies, a far lower number than the entire dataset at Kaggle which exceeds 134 000 plots. If the dataset was not restricted, then there would likely have been an increase in accuracy.

[1] See dataset genre distribution; limited space required this link.

[2] See word counts in plot descriptions; limited space required this link.

A careful look at the confusion matrix reveals that mislabeling took place mostly across drama/comedy lines. This task might be difficult even for human classifiers, we present some examples:

`Plot: (...) One day, he receives a surprise assignment: to fly over 12,000 kilometers of enemy waters to protect a girl named Juana del Moral. This mission is actually a covert operation as the original escort was taken down by the enemy. (...)`

`Ground-truth`: animation — `Predicted`: action

It is very challenging to know whether a movie is animated only by looking at its plot.

`Plot: A mother and wife stricken with memory loss allows a dysfunctional family a second chance at harmony and happiness`

`Ground-truth`: comedy — `Predicted`: drama

Differentiating between comedy and drama can be incredibly difficult, since drama can be used as a source of comedy.

`Plot: Satyameva Jayate is the story of a traffic constable who witnesses a murder committed by a politician's son in day light and the reactions he had to face when he tries to bring the culprits to book. As a bunch of dishonest bureaucrats and corrupted cops acts against him, he had to turn into the long-haired, local goon Pandiya to take revenge.`

`Ground-truth`: romance — `Predicted`: action

This movie is a romance, but there is no hint of it in the plot synopsis.

`Plot: A happily married couple's life is turned upside down when they are approached by a woman also claiming to be the husband's wife.`

`Ground-truth`: romance — `Predicted`: drama

Many romance movie synopses can be interpreted as being drama and vice-versa, since movies are very often both.

`Plot: A respectable woman's position in society is threatened by a blackmailer.`

`Ground-truth`: drama — `Predicted`: crime

Some plot synopses are incredibly short and can be interpreted as being part of a multitude of genres.

4 Future Work

If given more time, we plan to address the class imbalance in our dataset by creating custom datasets that give more weight to labels with fewer examples, while reducing the influence of those that are more frequent. We also aim to explore data augmentation techniques to achieve a more balanced dataset.

There is a potential in further tuning our TinyBERT model. Although it came close to being our best model, with improved fine-tuning, we believe it could yield even better results. One possible approach would be to freeze some of the initial layers and then train the entire architecture [4].

Another area of interest is the fact that movies often belong to multiple genres. So we would like to explore strategies for addressing cases where a movie has more than one genre label.

References

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint*, 2018.

[2] Ning Fei and Yangyang Zhang. Movie genre classification using tf-idf and svm. In *Proceedings of the 2019 7th international conference on information technology: IoT and smart city*, pages 131–136, 2019.

[3] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. Tinybert: Distilling bert for natural language understanding. *arXiv preprint*, 2019.

[4] Yufeng Zhao, Evelyn Soerjodjojo, and Haiying Che. Methods to enhance bert in aspect-based sentiment classification. In *2022 Euro-Asia Conference on Frontiers of Computer Science and Information Technology (FCSIT)*, pages 21–27. IEEE, 2022.